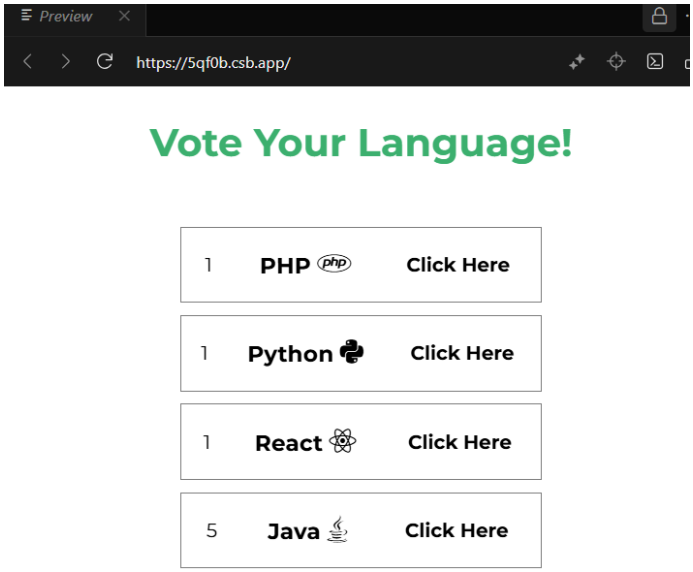
Old Code - Voting System

JavaScript - REACT component

```
import React, { useState, useCallback } from "react";

const functions = new Set();

const Voting = () => {
  const [count1, setCount1] = useState(0);
  const [count2, setCount2] = useState(0);
  const [count3, setCount3] = useState(0);
  const [count4, setCount4] = useState(0);

  const increment1 = useCallback(() => setCount1(count1 + 1), [count1]);
  const increment2 = useCallback(() => setCount2(count2 + 1), [count2]);
  const increment3 = useCallback(() => setCount3(count3 + 1), [count3]);
  const increment4 = useCallback(() => setCount4(count4 + 1), [count4]);

  functions.add(increment1);
  functions.add(increment2);
  functions.add(increment3);
  functions.add(increment4);

  return (
    <>
      <h2>Vote Your Language!</h2>
      <div className="vote-card">
        <div className="card-item">
          {count1}
          <h3>
            PHP<i className="fab fa-php"></i>
          </h3>
          <button className="btn btn-php" onClick={increment1}>
            Click Here
          </button>
        </div>
        <div className="card-item">
          {count2}
          <h3>
            Python<i className="fab fa-python"></i>
          </h3>
          <button className="btn btn-python" onClick={increment2}>
            Click Here
          </button>
        </div>
        <div className="card-item">
          {count3}
```

```
        <h3>
          React<i className="fab fa-react"></i>
        </h3>
        <button className="btn btn-react" onClick={increment3}>
          Click Here
        </button>
      </div>
      <div className="card-item">
        {count4}
        <h3>
          Java<i className="fab fa-java"></i>
        </h3>
        <button className="btn btn-java" onClick={increment4}>
          Click Here
        </button>
      </div>
    </div>
  </>
 );
};

export default Voting
```



CSS:

```
@import
url('https://fonts.googleapis.com/css2?family=Montserrat:wght@400;700&display=swap'
);

.App {
  font-family: 'Montserrat', sans-serif;
  text-align: center;
}
body {
  margin: 0 auto;
}
h2 {
  font-weight: bold;
  font-size: 2em;
  color: #3cb371;
}
.vote-card {
  margin-top: 50px;
}
.card-item {
  border: 1px solid grey;
  width: 50%;
  margin: 10px auto;
  display: flex;
  justify-content: space-around;
  align-items: center;
}
.btn {
  cursor: pointer;
  font-family: 'Montserrat', sans-serif;
  font-weight: bold;
  font-size: 1em;
  border: none;
  outline: none;
  background-color: white
}
.btn-php:hover {
  color: #784f9e;
}
.btn-python:hover {
  color: #f0e000;
}
.btn-react:hover {
  color: #44c0f1;
}
.btn-java:hover {
```

```
  color: #ff895b;
}
.fab {
  margin-left: 5px;
  font-size: 1.2em;
}
```

New enhanced voting System

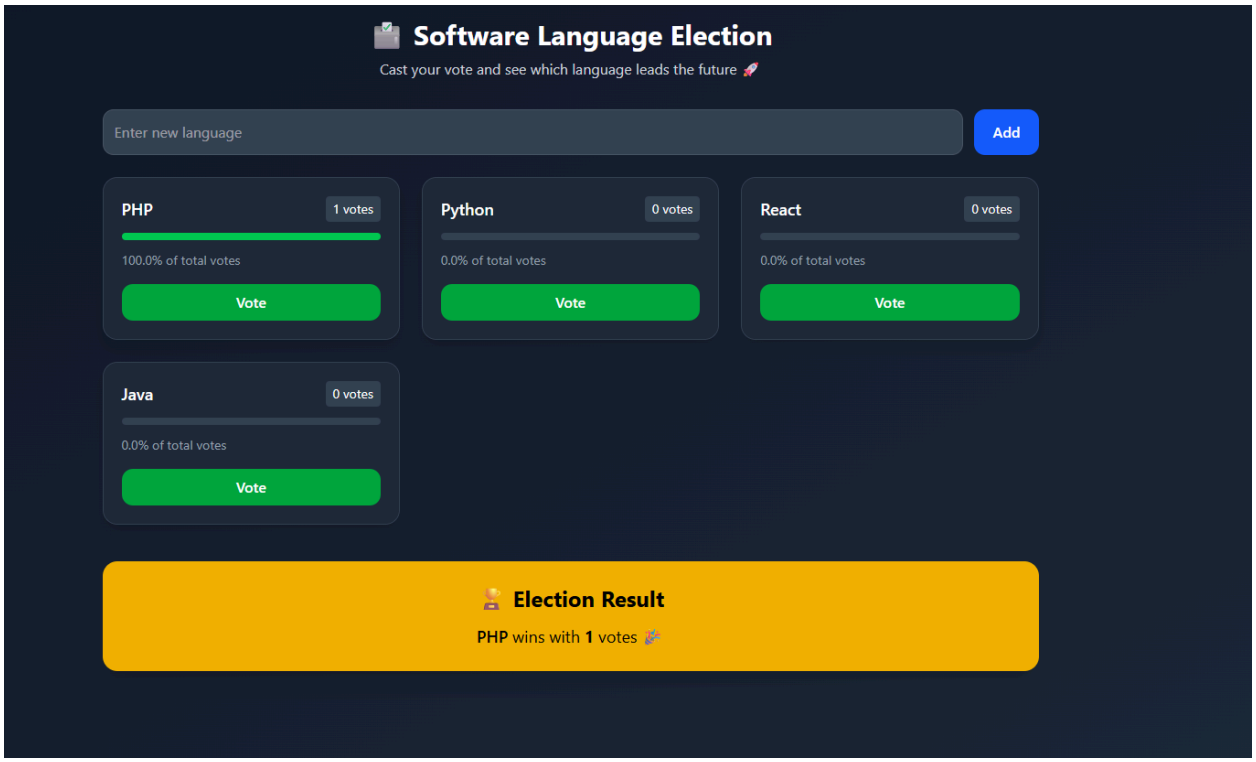
```
import React, { useState, useCallback, useMemo } from "react";

const Voting = () => {
  const [languages, setLanguages] = useState([
    { name: "PHP", votes: 0 },
    { name: "Python", votes: 0 },
    { name: "React", votes: 0 },
    { name: "Java", votes: 0 },
  ]);

  const [newLang, setNewLang] = useState("");

  const totalVotes = useMemo(
    () => languages.reduce((sum, l) => sum + l.votes, 0),
    [languages]
  );

  // Increment vote
  const incrementVote = useCallback((index) => {
    setLanguages((prev) =>
      prev.map((lang, i) =>
        i === index ? { ...lang, votes: lang.votes + 1 } : lang
      )
    );
  }, []);

  // Add new language
  const addLanguage = () => {
    if (!newLang.trim()) return;

    setLanguages((prev) => [
      ...prev,
      { name: newLang.trim(), votes: 0 },
    ]);
```

```
    setNewLang("");
  };

  // Compute winner
  const winner = useMemo(() => {
    const maxVotes = Math.max(...languages.map((l) => l.votes));
    if (maxVotes === 0) return null;

    return languages.filter((l) => l.votes === maxVotes);
  }, [languages]);

  return (
    <div className="min-h-screen bg-gradient-to-br from-gray-900 to-gray-800
text-white p-6">
      <div className="max-w-5xl mx-auto">
        {/* Header */}
        <div className="text-center mb-8">
          <h1 className="text-3xl font-bold">🗳️ Software Language Election</h1>
          <p className="text-gray-300 mt-2">
            Cast your vote and see which language leads the future 🚀
          </p>
        </div>

        {/* Add Language */}
        <div className="flex gap-3 mb-6">
          <input
            type="text"
            value={newLang}
            onChange={(e) => setNewLang(e.target.value)}
            placeholder="Enter new language"
            className="flex-1 p-3 rounded-xl bg-gray-700 border border-gray-600
focus:outline-none focus:ring-2 focus:ring-blue-500"
          />
          <button
            onClick={addLanguage}
            className="bg-blue-600 hover:bg-blue-700 px-5 py-3 rounded-xl font-semibold
shadow"
          >
            Add
          </button>
        </div>

        {/* Voting Grid */}
        <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-6">
          {languages.map((lang, index) => {
            const percentage = totalVotes
              ? ((lang.votes / totalVotes) * 100).toFixed(1)
              : 0;

            return (
```

```
            <div
              key={index}
              className="bg-gray-800 rounded-2xl p-5 shadow-lg border border-gray-700
hover:scale-[1.02] transition"
            >
              <div className="flex justify-between items-center mb-3">
                <h3 className="text-lg font-semibold">{lang.name}</h3>
                <span className="text-sm bg-gray-700 px-2 py-1 rounded">
                  {lang.votes} votes
                </span>
              </div>

              {/* Progress Bar */}
              <div className="w-full bg-gray-700 h-2 rounded mb-3">
                <div
                  className="bg-green-500 h-2 rounded"
                  style={{ width: `${percentage}%` }}
                />
              </div>

              <p className="text-sm text-gray-400 mb-4">
                {percentage}% of total votes
              </p>

              <button
                onClick={() => incrementVote(index)}
                className="w-full bg-green-600 hover:bg-green-700 py-2 rounded-xl
font-medium"
              >
                Vote
              </button>
            </div>
          );
        })}
      </div>

      {/* Result Section */}
      {winner && (
        <div className="mt-10 bg-yellow-500 text-black p-6 rounded-2xl shadow-xl
text-center">
          <h2 className="text-2xl font-bold mb-3">🏆 Election Result</h2>

          {winner.length === 1 ? (
            <p className="text-lg">
              <span className="font-semibold">{winner[0].name}</span> wins with{' '}
              <span class="font-bold">{winner[0].votes}</span> votes 🎉
            </p>
          ) : (
            <p className="text-lg">
              It's a tie between{' '}
```

```
            <span className="font-semibold">
              {winner.map((w) => w.name).join(", ")}
            </span>{' '}
            with {winner[0].votes} votes each 🤝
          </p>
        )}
      </div>
    )}
  </div>
</div>
);
};

export default Voting;
```

Output



AI Tools : Chatgpt
PlatForm : REACT

Enhancement Note:

Previously, the application supported only a fixed set of default languages, where users could vote by clicking a button and the vote count would increment accordingly.

With the new enhancements, users can now add their own programming languages dynamically and participate in the voting process. Additionally, the system displays vote percentages in real time, reflecting how each language compares with others based on total votes.

One of the major improvements is the introduction of a result section, which automatically determines and displays the winning language based on the highest number of votes.

Prompts

1)

Act as a senior React frontend developer.

Refactor my voting component to make it scalable and maintainable.
- Replace multiple useState hooks with a single state array
- Remove repetitive functions
- Use best practices (functional updates, clean structure)
- Keep the UI simple but structured

Return clean and production-ready React code.

2)
Enhance the voting system to allow users to dynamically add new programming languages.

Requirements:
- Add an input field and button
- On submit, add a new language with 0 votes
- Prevent empty inputs
- Update UI dynamically

Keep the code optimized and reusable.

3)

Enhance the voting UI to show real-time vote percentages.

Requirements:
- Calculate total votes
- Show percentage for each language
- Add a progress bar UI for visualization
- Update instantly when votes change

Use React hooks efficiently (useMemo if needed).

4)
Add a result section to the voting system.

Requirements:
- Determine the language with the highest votes
- Handle tie cases
- Do not show result if all votes are zero
- Display result in a highlighted card

Make the logic efficient and clean.

5)

Improve the UI of the voting system to look like a professional election dashboard.

Requirements:
- Use modern card-based layout
- Add shadows, spacing, and hover effects
- Include progress bars and percentages
- Add a highlighted result section
- Use a clean color theme (dark/light)

Focus on UX and visual hierarchy.

Productivity Obeservation

The enhanced voting system improves productivity by reducing repetitive code and making the component scalable with dynamic data handling.
Real-time updates and automated result calculation provide instant insights, improving user experience and decision-making efficiency.